

Praktikum 4 — Probability and Statistical Inference

Data Science · Magister Ilmu Komputer, FMIPA UGM Companion to **Chapter 4**. Assignment weight: **3%**.

What you will do

1. Apply Bayes' theorem and feel the base-rate effect in natural frequencies.
2. Watch the central limit theorem converge — and fail.
3. Build confidence intervals, including a bootstrap for a statistic with no formula.
4. Run a complete two-group comparison: assumptions, test, effect size, CI, power.
5. Correct for multiple comparisons.
6. Analyse a real A/B test end to end, and simulate the cost of peeking.

The dataset. `data/ab_test.csv` — 70,000 sessions from a two-week experiment on a checkout page.

```
import sys
from pathlib import Path

ROOT = Path.cwd()
while not (ROOT / "checks.py").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent
sys.path.insert(0, str(ROOT))

import checks
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt

DATA = ROOT / "data"
plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False,
                    "axes.spines.right": False, "axes.grid": True,
                    "grid.alpha": 0.3, "axes.titleweight": "bold"})
```

1. Bayes and the base rate (DEMO)

A fraud detector catches 92% of frauds and has a false-positive rate of 1.5%. Fraud occurs in 0.3% of transactions. A transaction is flagged. What is the probability it is genuinely fraudulent?

$$P(F \mid +) = \frac{P(+ \mid F)P(F)}{P(+ \mid F)P(F) + P(+ \mid F^c)P(F^c)}$$

```

sens, fpr, prev = 0.92, 0.015, 0.003

p_pos = sens * prev + fpr * (1 - prev)
p_fraud_given_pos = sens * prev / p_pos

print(f"P(+)      = {p_pos:.6f}")
print(f"P(F | +)  = {p_fraud_given_pos:.4f}")

# Natural frequencies make the same result intuitive.
N = 100_000
print(f"\nOf {N:,} transactions:")
print(f"  {N*prev:>8,.0f} are fraudulent, of which {N*prev*sens:>7,.0f} are flagged")
print(f"  {N*(1-prev)>8,.0f} are legitimate, of which {N*(1-prev)*fpr:>7,.0f} are flagged")
print(f"  {N*p_pos:>8,.0f} flags in total, of which only "
      f"{N*prev*sens/(N*p_pos):.1%} are genuine")

```

The false positives dominate because the legitimate group is 332 times larger. This is not a defect of the detector — it is arithmetic, and it is the reason accuracy is meaningless on imbalanced data.

TASK 4.1

Report $P(F \mid +)$ as a proportion.

```

p_fraud = None

checks.task("4.1", p_fraud)

```

2. The central limit theorem (DEMO)

The CLT says the sampling distribution of the **mean** approaches normality regardless of the population's shape, with standard error $\sigma/\sqrt{n}$.

Two things it does **not** say: that individual observations become normal, and that $n \geq 30$ is always enough.

```

rng = np.random.default_rng(3)
pop = rng.exponential(scale=2.0, size=1_000_000)      # skewness = 2

print(f"population: mean={pop.mean():.3f}  sd={pop.std():.3f}  "
      f"skew={float(stats.skew(pop)):.3f}\n")

rows = []
for n in (2, 5, 30, 200):
    means = rng.choice(pop, size=(20_000, n)).mean(axis=1)
    rows.append({"n": n, "mean_of_means": means.mean(),
                 "observed_se": means.std(),
                 "sigma_over_sqrt_n": pop.std() / np.sqrt(n),
                 "skew": float(stats.skew(means))})
clt = pd.DataFrame(rows)
print(clt.round(4).to_string(index=False))

```

The observed spread of the sample means tracks $\sigma/\sqrt{n}$ closely at every n — the standard error formula is exact, not asymptotic. But the *skewness* falls only gradually: still 0.37 at $n=30$. The familiar rule of thumb is optimistic for skewed populations.

```
fig, axes = plt.subplots(1, 4, figsize=(14, 3))
for ax, n in zip(axes, (2, 5, 30, 200)):
    means = rng.choice(pop, size=(20_000, n)).mean(axis=1)
    ax.hist(means, bins=60, density=True, color="#2E6F9E",
            edgecolor="white", linewidth=0.2)
    xs = np.linspace(means.min(), means.max(), 200)
    ax.plot(xs, stats.norm.pdf(xs, pop.mean(), pop.std() / np.sqrt(n)),
            color="#9C3535", lw=1.5)
    ax.set(title=f"n = {n} (skew {float(stats.skew(means)):.2f})", yticks=[])
fig.suptitle("Sampling distribution of the mean, exponential population",
            y=1.04, fontweight="bold")
plt.show()
```

```
# Where the CLT fails: the Cauchy distribution has no finite mean or variance,
# so no amount of averaging helps.
cauchy_means = np.array([rng.standard_cauchy(200).mean() for _ in range(2000)])
normal_means = np.array([rng.normal(0, 1, 200).mean() for _ in range(2000)])
print(f"Cauchy, n=200 : sd of sample means = {cauchy_means.std():10.3f}")
print(f"Normal, n=200 : sd of sample means = {normal_means.std():10.3f}"
      f" (theory {1/np.sqrt(200):.3f})")
print("\nThe CLT requires finite variance. Check its conditions before invoking it.")
```

3. Confidence intervals and the bootstrap (DEMO)

A 95% CI is a statement about the **procedure**: under repeated sampling, 95% of such intervals contain the true parameter. It is *not* a 95% probability that this particular interval does.

When a statistic has no tractable sampling distribution — a median, a correlation, a ratio — resample instead.

```
ab = pd.read_csv(DATA / "ab_test.csv")
print(ab.shape)
ab.head()
```

```
secs = ab.loc[ab["variant"] == "A", "session_seconds"].to_numpy()

# 1. t-interval for the MEAN (valid, but answers a different question)
m, se = secs.mean(), stats.sem(secs)
lo, hi = stats.t.interval(0.95, len(secs) - 1, loc=m, scale=se)
print(f"mean {m:7.2f} t-interval [{lo:7.2f}, {hi:7.2f}]")

# 2. Percentile bootstrap for the MEDIAN
B = 5000
boot = np.median(rng.choice(secs, size=(B, len(secs)), replace=True), axis=1)
blo, bhi = np.percentile(boot, [2.5, 97.5])
print(f"median {np.median(secs):7.2f} bootstrap CI [{blo:7.2f}, {bhi:7.2f}]")

# 3. BCa bootstrap, which corrects for bias and skew
res = stats.bootstrap((secs,), np.median, confidence_level=0.95,
                      n_resamples=5000, method="BCa", random_state=0)
print(f"BCa interval "
      f"[{res.confidence_interval.low:7.2f}, {res.confidence_interval.high:7.2f}]")

print("\nThe mean and the median are different quantities on skewed data.")
print("Choosing which to report is a substantive decision, not a technical one.")
```

4. A complete two-group comparison (DEMO)

The full reporting discipline: assumptions, test, **effect size**, confidence interval, power. A p-value on its own is a far weaker statement.

```
a = ab.loc[ab["variant"] == "A", "session_seconds"].to_numpy()
b = ab.loc[ab["variant"] == "B", "session_seconds"].to_numpy()

# 1. Assumptions (on a subsample -- Shapiro-Wilk is not meaningful at n=35,000)
sub = rng.choice(a, 500, replace=False), rng.choice(b, 500, replace=False)
print(f"Shapiro A p = {stats.shapiro(sub[0]).pvalue:.4f} "
      f"| Shapiro B p = {stats.shapiro(sub[1]).pvalue:.4f}")
print(f"Levene p = {stats.levene(a, b).pvalue:.4f}")
print("(Session times are log-normal, so normality is rejected. At this n the")
print("CLT makes the t-test robust anyway -- but check, do not assume.)\n")

# 2. Test: Welch by default. Note the sign -- ttest_ind(a, b) tests a - b.
t, p = stats.ttest_ind(a, b, equal_var=False)
print(f"Welch t = {t:.3f}, p = {p:.3e}")

# 3. Effect size (Cohen's d, pooled sd)
n1, n2 = len(a), len(b)
sp = np.sqrt(((n1 - 1) * a.var(ddof=1) + (n2 - 1) * b.var(ddof=1)) / (n1 + n2 - 2))
d = (b.mean() - a.mean()) / sp
print(f"difference = {b.mean() - a.mean():.2f} s, Cohen's d = {d:.3f}")

# 4. CI for the difference (Welch-Satterthwaite df)
se_d = np.sqrt(a.var(ddof=1) / n1 + b.var(ddof=1) / n2)
df_w = se_d**4 / ((a.var(ddof=1)/n1)**2/(n1-1) + (b.var(ddof=1)/n2)**2/(n2-1))
clo, chi = stats.t.interval(0.95, df_w, loc=b.mean() - a.mean(), scale=se_d)
print(f"95% CI for the difference: [{clo:.2f}, {chi:.2f}] s")

# 5. Achieved power (statsmodels-free, via the non-central t)
ncp = d * np.sqrt(n1 * n2 / (n1 + n2))
crit = stats.t.ppf(0.975, n1 + n2 - 2)
power = (1 - stats.nct.cdf(crit, n1 + n2 - 2, ncp)
        + stats.nct.cdf(-crit, n1 + n2 - 2, ncp))
print(f"achieved power = {power:.3f}")
```

The sentence to write:

Sessions on variant B lasted 3.5 s longer on average (95% CI 2.4 to 4.7 s; Welch $t = 6.15$, $p < 0.001$; Cohen's $d = 0.047$).

The negative sign on t merely reflects that `ttest_ind(a, b)` tests $\bar{a} - \bar{b}$; read the direction from the group means, not from the sign of the statistic.

Note that Cohen's d is **0.047** — a trivially small effect that is overwhelmingly significant, because $n = 70\{, \}000$. **Statistical significance is not practical significance.** Always report the effect size.

5. Categorical association (DEMO)

```
table = pd.crosstab(ab["variant"], ab["converted"])
print(table.to_string())

chi2, p_chi, dof, expected = stats.chi2_contingency(table)
n_tot = table.to_numpy().sum()
cramers_v = np.sqrt(chi2 / (n_tot * (min(table.shape) - 1)))

print(f"\nchi2 = {chi2:.3f}, dof = {dof}, p = {p_chi:.3e}")
print(f"Cramer's V = {cramers_v:.4f}    <- the effect size: tiny but real")
```

6. Multiple comparisons (DEMO)

Testing m independent hypotheses at $\alpha = 0.05$ when all nulls are true gives a probability $1 - 0.95^m$ of at least one false rejection: 40% at $m = 10$, 99% at $m = 90$.

```
for m in (1, 5, 10, 20, 50, 90):
    print(f"m = {m}>3: P(at least one false positive) = {1 - 0.95**m:.3f}")
```

```
def adjust(pvals, method):
    """Bonferroni / Holm / Benjamini-Hochberg, without statsmodels."""
    p = np.asarray(pvals, dtype=float)
    m = len(p)
    if method == "bonferroni":
        return np.minimum(p * m, 1.0)
    if method == "holm":
        order = np.argsort(p)
        adj = np.empty(m)
        run = 0.0
        for i, idx in enumerate(order):
            run = max(run, (m - i) * p[idx])
            adj[idx] = min(run, 1.0)
        return adj
    if method == "fdr_bh":
        order = np.argsort(p)[::-1]
        adj = np.empty(m)
        run = 1.0
        for i, idx in enumerate(order):
            rank = m - i
            run = min(run, p[idx] * m / rank)
            adj[idx] = run
        return adj
    raise ValueError(method)

pvals = np.array([0.001, 0.008, 0.021, 0.033, 0.041, 0.049,
                  0.30, 0.44, 0.62, 0.81])
print(f"raw p < 0.05: {(pvals < 0.05).sum()} of {len(pvals)}\n")
for method in ("bonferroni", "holm", "fdr_bh"):
    adj = adjust(pvals, method)
    print(f"{method:<12} rejects {(adj < 0.05).sum()} of {len(pvals)}    "
          f"adj = {np.round(adj, 4)}")
```

Six raw p-values were below 0.05. Controlling the **family-wise error rate** (Bonferroni, Holm) leaves one survivor; controlling the **false discovery rate** (BH) leaves two.

Neither is wrong — they answer different questions. FWER asks *"what is the chance I make any false claim?"*, and suits confirmatory work. FDR asks *"what fraction of my claims will be false?"*, and suits screening. What is **not** defensible is reporting all six as discoveries.

Note that Holm dominates Bonferroni everywhere, so plain Bonferroni is never the better choice.

7. Analysing the A/B test (DEMO + TASK)

Step 0 — checks *before* the primary metric

A sample-ratio mismatch means randomisation or logging is broken, and no result from the test can be trusted. Check it every time.

```
counts = ab["variant"].value_counts().sort_index()
chi2_srm, p_srm = stats.chisquare(counts.to_numpy())
print("Sample ratio check")
print(counts.to_string())
print(f" observed split: {counts.iloc[1] / counts.sum():.4f}")
print(f" chi2 p = {p_srm:.3f} -> "
      f"{'OK' if p_srm > 0.01 else 'BROKEN, stop here'}\n")

print("Balance on a pre-treatment covariate (device):")
dev = pd.crosstab(ab["variant"], ab["device"], normalize="index")
print(dev.round(4).to_string())
print(f" chi2 p = {stats.chi2_contingency(pd.crosstab(ab['variant'], ab['device']))[1]:.3f}")
```

Step 1 — the primary metric

```
grp = ab.groupby("variant")["converted"].agg(["sum", "count", "mean"])
print(grp.round(5).to_string())

conv = grp["sum"].to_numpy()
trials = grp["count"].to_numpy()
r1, r2 = conv / trials
pool = conv.sum() / trials.sum()

se_pool = np.sqrt(pool * (1 - pool) * (1 / trials[0] + 1 / trials[1]))
z = (r2 - r1) / se_pool
p_two = 2 * (1 - stats.norm.cdf(abs(z)))

se_unpooled = np.sqrt(r1 * (1 - r1) / trials[0] + r2 * (1 - r2) / trials[1])
diff = r2 - r1

print(f"\ncontrol {r1:.4%} variant {r2:.4%}")
print(f"absolute lift {diff:+.4%} "
      f"(95% CI [{diff - 1.96*se_unpooled:+.4%}, {diff + 1.96*se_unpooled:+.4%}])")
print(f"relative lift {diff / r1:+.2%}")
print(f"z = {z:.3f}, p = {p_two:.6f}")
```

Step 2 — was the test big enough?

The sample size should have been computed **before** the experiment, from the minimum detectable effect. Here is that calculation, after the fact:

```
def n_per_arm(p1, rel_mde, alpha=0.05, power=0.80):
    """Sample size per arm to detect a relative lift, via Cohen's h."""
    p2 = p1 * (1 + rel_mde)
    h = abs(2 * np.arcsin(np.sqrt(p2)) - 2 * np.arcsin(np.sqrt(p1)))
    z_a = stats.norm.ppf(1 - alpha / 2)
    z_b = stats.norm.ppf(power)
    return int(np.ceil(((z_a + z_b) / h) ** 2))

for mde in (0.05, 0.10, 0.12, 0.20):
    n_req = n_per_arm(r1, mde)
    verdict = "adequately powered" if trials[0] >= n_req else "UNDERPOWERED"
    print(f"to detect a {mde:>4.0%} relative lift: {n_req:>7,} per arm  "
          f"(we have {trials[0]:,}) -> {verdict}")

print("\nHalving the effect you want to detect quadruples the sample required:")
print("the denominator is (p2 - p1)^2.")
```

TASK 4.2

Report the two-sided p-value for the difference in conversion rate.

```
p_konversi = None

checks.task("4.2", p_konversi)
```

TASK 4.3

Report the **absolute** lift (as a proportion, e.g. 0.0072 for 0.72 points).

```
lift_absolut = None

checks.task("4.3", lift_absolut)
```

8. The cost of peeking (DEMO + TASK)

Checking the p-value repeatedly and stopping when it crosses 0.05 inflates the false-positive rate dramatically. Here we simulate it on data where the null is **true by construction**, so every "significant" result is a false positive.

```
def peeking_experiment(n_max=2000, step=50, n_sims=400, seed=0):
    """Return (rate with peeking, rate testing once at the end)."""
    rng = np.random.default_rng(seed)
    peeked = fixed = 0
    checkpoints = range(step, n_max + 1, step)
    for _ in range(n_sims):
        x = rng.normal(0, 1, n_max)
        y = rng.normal(0, 1, n_max)          # SAME distribution: null is true
        hit = False
        for k in checkpoints:
            if stats.ttest_ind(x[:k], y[:k]).pvalue < 0.05:
                hit = True
                break
        peeked += hit
        fixed += stats.ttest_ind(x, y).pvalue < 0.05
    return peeked / n_sims, fixed / n_sims

rate_peek, rate_fixed = peeking_experiment()
print(f"stopping at the first significant look : {rate_peek:.1%}")
print(f"testing once, at the planned end      : {rate_fixed:.1%}")
print(f"nominal alpha                        : 5.0%")
print("\nThe null is true in every simulation. Peeking manufactures")
print("false positives out of nothing.")
```

TASK 4.4

Report the false-positive rate under peeking (as a proportion).

```
laju_peeking = None

checks.task("4.4", laju_peeking)
```

9. Take-home (assessed)

A. A Poisson call centre. A centre receives on average 8 calls per hour. Compute (a) $P(X = 0)$ in an hour; (b) $P(X \geq 12)$; (c) the probability the wait to the next call exceeds 15 minutes. State which distribution you used for (c) and why it is not the Poisson.

B. Interpret a confidence interval. A 95% CI for a mean difference is $[-0.4, 6.8]$. Which of these are correct, and correct those that are not? (a) "There is a 95% probability the true difference is between -0.4 and 6.8 ." (b) "We cannot reject H_0 at $\alpha = 0.05$." (c) "The effect is probably zero." (d) "A larger sample would produce a narrower interval."

C. Design an A/B test. A checkout redesign will ship only for a relative lift of at least 8% on a 2.8% baseline. Compute the required sample size per arm at 80% power, state the expected duration at 12,000 sessions/day, and list **three checks** you would run before analysing the primary metric.

D. Segment analysis, done honestly. Split the A/B result by `device`. Report the lift in each segment with a confidence interval, apply an appropriate multiplicity correction, and answer: *does variant B work better on mobile than on desktop, or is the difference within noise?* Be explicit about how many tests you performed in total.

E. What is missing. An analyst reports: "The variant increased revenue per user by 3.2% ($p = 0.03$), so we should ship it." Identify **four** pieces of information missing from that statement and say what each would tell you.

Next week: turning raw columns into features a model can actually learn from.